

C++ 프로그래밍

템플릿 (Template)

코드 중복 문제

타입만 다르고 로직이 같은 함수를 **타입별로 따로** 작성해야 할까?

타입별로 각각 작성

```
void swap(int& a, int& b) {  
    int tmp = a;  
    a = b;  
    b = tmp;  
}  
  
void swap(double& a, double& b) {  
    double tmp = a;  
    a = b;  
    b = tmp;  
}  
  
void swap(std::string& a, std::string& b) {  
    std::string tmp = a;  
    a = b;  
    b = tmp;  
}
```

문제점

- 로직이 **완전히 동일**하지만 타입만 다름
- 새 타입이 생길 때마다 **함수를 추가**해야 함
- 한 곳의 버그가 **모든 함수에 복사**됨
- 코드 유지보수 부담 증가

해결책: 타입을 매개변수처럼 다루는 템플릿

함수 템플릿 — 기본 문법

`template<typename T>` 로 타입을 매개변수처럼 받으면 **하나의 코드로** 모든 타입을 처리한다.

함수 템플릿 선언

```
template<typename T>      // T: 타입 매개변수
void swap(T& a, T& b) {
    T tmp = a;
    a = b;
    b = tmp;
}
```

사용 — 타입 자동 추론

```
int x = 1, y = 2;
swap(x, y);           // T = int 로 추론
// x=2, y=1

double p = 3.14, q = 2.72;
swap(p, q);           // T = double 로 추론
// p=2.72, q=3.14

std::string s = "hello", t = "world";
swap(s, t);           // T = std::string 으로 추론
// s="world", t="hello"
```

컴파일러가 인수 타입을 보고 `T` 를 **자동으로 결정**한다.

키워드

역할

`template`

템플릿 선언 시작

`typename T`

타입 매개변수 이름 선언

`T`

실제 타입의 자리표시자

class T VS typename T

두 키워드는 **문법적으로 동일**하지만, `typename` 이 더 정확하고 현대적인 표현이다.

둘 다 컴파일된다

```
// class 키워드 - 초기 C++의 방식
template<class T>
void swap(T& a, T& b) {
    T tmp = a; a = b; b = tmp;
}
```

```
// typename 키워드 - C++98 추가, 권장
template<typename T>
void swap(T& a, T& b) {
    T tmp = a; a = b; b = tmp;
}
```

- 두 선언은 **완전히 동등**하다
- 어떤 타입이든 전달 가능 (`int` , `double` , 클래스 모두)
- `class` 는 클래스 타입만 받을 것 같은 **오해**를 준다.

typename T 사용을 권장

```
template<class T>    // ← T가 클래스여야만 할 것 같다
T max_val(T a, T b) { return a > b ? a : b; }

max_val(3, 5);      // T = int    - 기본 타입도 OK
max_val(3.14, 2.72); // T = double - 기본 타입도 OK
```

`typename` 은 "**어떤 타입이든**" 이라는 의도를 명확히 전달.

	class T	typename T
등장	초기 C++ (표준 이전)	C++98 정식 추가
누양스	클래스 타입만?	어떤 타입이든
권장 여부	❌ 구식	✅ 현대 C++ 권장
표준 라이브러리 소스와 현대 C++ 코드는 <code>typename</code> 을 사용한다.		

인스턴스화 (Instantiation)

템플릿은 설계도다. 컴파일러가 구체적인 타입을 대입해 실제 함수를 자동 생성한다.

컴파일러가 생성하는 코드

```
// 원본 템플릿 (설계도)
template<typename T>
void swap(T& a, T& b) {
    T tmp = a; a = b; b = tmp;
}

// T = int 로 호출 시 → 컴파일러 자동 생성
void swap(int& a, int& b) {
    int tmp = a; a = b; b = tmp;
}

// T = double 로 호출 시 → 컴파일러 자동 생성
void swap(double& a, double& b) {
    double tmp = a; a = b; b = tmp;
}
```

타입 추론 vs 명시적 지정

```
template<typename T>
T max_val(T a, T b) {
    return a > b ? a : b;
}

// 타입 추론 - 인수로 T 자동 결정
max_val(3, 5);           // T = int
max_val(3.14, 2.72);     // T = double

// 명시적 지정 - <> 안에 T를 직접 기입
max_val<int>(3, 5);       // T = int
max_val<double>(3, 5);    // T = double
                        // (3, 5가 double로 변환됨)
```

여러 템플릿 매개변수

템플릿 매개변수는 **타입**(`typename T`) 뿐만 아니라 **정수 값**(`int N`) 도 될 수 있다.

두 타입을 받는 함수 템플릿

```
#include <iostream>

template<typename T, typename U>
void print_pair(T a, U b) {
    std::cout << a << ", " << b << "\n";
}

int main() {
    print_pair(42, "hello");    // T=int,    U=const char*
    print_pair(3.14, true);    // T=double, U=bool
    print_pair('A', 65);       // T=char,   U=int
}

42, hello
3.14, 1
A, 65
```

타입 + 값을 함께 받는 함수 템플릿

```
#include <iostream>

// T: 타입 매개변수, N: 정수 값 매개변수
template<typename T, int N>
void repeat(T val) {
    for (int i = 0; i < N; i++)
        std::cout << val << "\n";
}

int main() {
    repeat<std::string, 3>("hello");
    // hello
    // hello
    // hello

    repeat<int, 2>(42);
    // 42
    // 42
}
```

N 은 컴파일 타임에 결정되는 **상수** — 런타임 변수는 전달할 수 없다.

클래스 템플릿 — 기본

`template<typename T>` 를 클래스 앞에 붙이면 **타입을 매개변수로 받는 클래스**를 만든다.

Pair 클래스 템플릿

```
template<typename T, typename U>
class Pair {
public:
    T first;
    U second;

    Pair(T f, U s) : first(f), second(s) {}

    void print() const {
        std::cout << first << ", " << second << "\n";
    }
};

// 사용 - 타입 인수를 <> 안에 명시
Pair<int, std::string> p1(1, "hello");
p1.print();    // 1, hello

Pair<double, bool> p2(3.14, true);
p2.print();    // 3.14, 1
```

함수 템플릿과의 차이

	함수 템플릿	클래스 템플릿
타입 추론	인수에서 자동 추론	직접 명시 필요
인스턴스화	<code>f(x)</code> 호출 시	변수 선언 시
예시	<code>max_val(3, 5)</code>	<code>Pair<int, int></code>

C++17부터 클래스 템플릿도 **생성자 인수**로 타입 추론 가능 (CTAD)

```
Pair p(1, "hello"); // C++17: Pair<int, const char*>
```

Stack 클래스 템플릿

클래스 템플릿으로 어떤 타입이든 담을 수 있는 스택을 만들 수 있다.

Stack<T> 구현

```
#include <vector>
#include <stdexcept>

template<typename T>
class Stack {
    std::vector<T> data;
public:
    void push(const T& v) { data.push_back(v); }

    void pop() {
        if (empty()) throw std::runtime_error("empty");
        data.pop_back();
    }

    T top() const {
        if (empty()) throw std::runtime_error("empty");
        return data.back();
    }

    bool empty() const { return data.empty(); }
    int size() const { return (int)data.size(); }
};
```

사용 — 타입만 바꿔 재사용

```
// int 스택
Stack<int> si;
si.push(1); si.push(2); si.push(3);
std::cout << si.top();    // 3
si.pop();
std::cout << si.top();    // 2

// string 스택 — 같은 클래스, 다른 타입
Stack<std::string> ss;
ss.push("hello");
ss.push("world");
std::cout << ss.top();    // world
```

Stack<int> 과 Stack<std::string> 은 별개의 클래스로 인스턴스화된다.

비타입 템플릿 매개변수 — Vector, Matrix

크기를 컴파일 타임 상수로 고정하는 `Vector` 와 `Matrix` 클래스를 구현한다.

Vector<T, N> 클래스

```
template<typename T, int N>
class Vector {
    T data[N] = {};           // N: 컴파일 타임 크기, 0 초기화
public:
    T& operator()(int i)      { return data[i]; }
    const T& operator()(int i) const { return data[i]; }
    int size() const { return N; }
};

// int형 3차원 벡터
Vector<int, 3> v;
v(0) = 1; v(1) = 2; v(2) = 3;

// float형 2차원 벡터
Vector<float, 2> vf;
vf(0) = 1.5f; vf(1) = 2.5f;
```

Matrix<T, R, C> 클래스

```
template<typename T, int R, int C> // R: 행, C: 열
class Matrix {
    T data[R][C] = {};
public:
    T& operator()(int r, int c)      { return data[r][c]; }
    const T& operator()(int r, int c) const { return data[r][c]; }
    int rows() const { return R; }
    int cols() const { return C; }
};

// int형 2x3 행렬
Matrix<int, 2, 3> A;
A(0,0)=1; A(0,1)=2; A(0,2)=3;
A(1,0)=4; A(1,1)=5; A(1,2)=6;
```

typedef / using — 긴 타입 이름에 별칭 붙이기

템플릿으로 만든 타입은 <> 인수가 붙어 길어진다. 별칭을 붙이면 더 간결하게 쓸 수 있다.

typedef — C++ 전통 방식

```
typedef Vector<int, 2>      Vec2i;
typedef Vector<float, 3>    Vec3f;
typedef Matrix<float, 4, 4> Mat4f;
typedef Stack<int>          IntStack;
```

using — C++11 타입 별칭 권장

```
using Vec2i    = Vector<int, 2>;
using Vec3f    = Vector<float, 3>;
using Mat4f    = Matrix<float, 4, 4>;
using IntStack = Stack<int>;
```

using 은 의미가 왼쪽 = 오른쪽 구조라 읽기 쉽고, typedef 로는 불가능한 템플릿 별칭도 만들 수 있다.

별칭을 쓰면 코드가 깔끔해진다

```
// 별칭 없이 사용할 경우, 복잡함.
Vector<int, 2> a, b;
Matrix<float, 4, 4> A;
```

```
// 별칭을 사용하면 코드가 깔끔함.
Vec2i x, y;
Mat4f M;
```

```
a(0) = 1; a(1) = 2;
b(0) = 3; b(1) = 4;
```

```
Vec3f v;
v(0) = 1.0f; v(1) = 2.0f; v(2) = 3.0f;
```

```
IntStack s;
s.push(10);
s.push(20);
std::cout << s.top();    // 20
```

별칭은 새 타입이 아닌 기존 타입의 다른 이름 — 원래 타입과 혼용 가능.

Matrix * Vector, Vector * Matrix 곱 — operator*

operator* 를 오버로딩해 인수 순서만으로 두 방향 곱을 모두 지원한다.

operator* 함수 템플릿 (오버로딩)

```
// 1) Matrix * Vector : RxC 행렬 * C차원 벡터 → R차원 벡터
template<typename T, int R, int C>
Vector<T, R> operator*(const Matrix<T, R, C>& A,
                       const Vector<T, C>& v) {
    Vector<T, R> result;
    for (int r = 0; r < R; r++)
        for (int c = 0; c < C; c++)
            result(r) += A(r, c) * v(c);
    return result;
}

// 2) Vector * Matrix : R차원 벡터 * RxC 행렬 → C차원 벡터
template<typename T, int R, int C>
Vector<T, C> operator*(const Vector<T, R>& v,
                       const Matrix<T, R, C>& A) {
    Vector<T, C> result;
    for (int c = 0; c < C; c++)
        for (int r = 0; r < R; r++)
            result(c) += v(r) * A(r, c);
    return result;
}
```

인수 순서(행렬.벡터 vs 벡터.행렬)만으로 컴파일러가 1),2)를 자동 선택.

using 별칭으로 깔끔하게

```
using Mat2x3i = Matrix<int, 2, 3>;
using Vec3i   = Vector<int, 3>;
using Vec2i   = Vector<int, 2>;
using Mat2x3f = Matrix<float, 2, 3>;
using Vec3f   = Vector<float, 3>;
using Vec2f   = Vector<float, 2>;

// — int형 —
Mat2x3i A;
A(0,0)=1; A(0,1)=2; A(0,2)=3;
A(1,0)=4; A(1,1)=5; A(1,2)=6;

Vec3i v;
v(0)=1; v(1)=2; v(2)=3;

Vec2i r = A * v;    // r(0) = 14,  r(1) = 32

// — float형 —
Mat2x3f B;
B(0,0)=0.5f; B(0,1)=1.0f; B(0,2)=1.5f;
B(1,0)=2.0f; B(1,1)=2.5f; B(1,2)=3.0f;

Vec3f vf;
vf(0)=1.0f; vf(1)=2.0f; vf(2)=3.0f;

Vec2f rf = B * vf;    // rf(0) = 7.0,  rf(1) = 16.0
```

정리

개념	문법	용도
함수 템플릿	<code>template<typename T> void f(T a)</code>	타입만 다른 함수를 하나로
클래스 템플릿	<code>template<typename T> class C { }</code>	타입만 다른 클래스를 하나로
타입 추론	<code>f(x)</code> — 인수에서 자동 결정	함수 템플릿에서 편리하게
명시적 지정	<code>f<int>(x)</code>	추론이 안 되거나 명확히 할 때
비타입 매개변수	<code>template<typename T, int N></code>	컴파일 타임 상수 값 전달
타입 별칭 (<code>using</code>)	<code>using Vec2i = Vector<int,2>;</code>	긴 템플릿 타입에 짧은 이름 (C++11 권장)
연산자 오버로딩	<code>operator*(A, v) → A * v</code>	템플릿과 결합해 자연스러운 표현
인스턴스화	컴파일러 자동 생성	각 타입 조합마다 별개 코드

핵심 원칙

- 템플릿은 **설계도** — 사용 시점에 컴파일러가 구체적인 코드를 생성한다
- STL의 모든 컨테이너·알고리즘이 **템플릿으로 구현**됨